VSCode 本地 C++ 环境配置（Windows / macOS / Linux）

已有 VSCode 的同学安装 g++ 后，在 VSCode 集成终端中检查版本、编译并运行 Hello World，即完成本环节。使用 C++17 标准。

尚未完成 VSCode 配置的同学，使用原来的 CLion 方案：CLion 中文指南 · CLion English guide。

1. 按操作系统安装 g++

本指南面向已经安装并能正常使用 VSCode 的同学。保留现有配置，按自己的操作系统安装编译器。**安装 VSCode 或 C/C++ 扩展不等于安装了 g++**。如果已经在 VSCode 终端中运行 `g++ -v` 成功，可以直接前往第 2 节的 Hello World 验收。

选择菜单 **终端 → 新建终端（Terminal → New Terminal）**，下方打开的 Terminal 就是本文执行检查、编译和运行命令的位置。不要在“输出（Output）”或“调试控制台（Debug Console）”中输入命令。Windows 使用 PowerShell，macOS/Linux 使用 zsh 或 bash。

Windows

参考视频：Windows 下的 C++ 环境配置教程。已安装 VSCode 的同学重点完成编译器安装和 PATH 配置，再回到本文验收。

以下给出 Windows x64 使用 MSYS2 安装 g++ 的步骤：

1. 从 MSYS2 官网 下载并安装 MSYS2。以下以默认安装目录 `C:\msys64` 为例。

2. 在开始菜单打开 **MSYS2 UCRT64** 终端，执行：

```
pacman -S --needed mingw-w64-ucrt-x86_64-gcc
```

出现确认提示时按回车或输入 `Y`，等待安装完成。这条安装命令在 MSYS2 UCRT64 中执行。

3. 在 Windows 搜索中打开“编辑账户的环境变量”，编辑用户变量 `Path`，新增 `C:\msys64\ucrt64\bin`。如果修改过安装位置，请填写对应的 `ucrt64\bin` 路径。

4. 保存环境变量，完全退出所有 VSCode 窗口，再重新打开。选择菜单 **终端 → 新建终端（Terminal → New Terminal）**，使用 PowerShell，执行：

```
g++ -v
```

能看到 GCC 版本和编译器信息，就继续完成下方的 Hello World。若提示找不到 g++，检查 PATH 是否指向包含 `g++.exe` 的目录，并确认已重启 VSCode。

macOS：安装 Xcode Command Line Tools

macOS 使用 Xcode Command Line Tools 提供的编译器。在 VSCode 的集成终端中执行：

```
xcode-select --install
```

按照弹窗提示完成安装，等待安装结束。如果提示开发工具已经安装，直接继续下一步。无需安装完整的 Xcode 应用。

在 VSCode 中新建终端，执行：

```
g++ -v
```

看到 Apple Clang 版本信息即正常：macOS 的 g++ 命令实际调用 Apple Clang，可以按本指南使用 `g++ -std=c++17` 编译课程程序，无需另外安装 GNU GCC。

Linux：先检查，再按需安装

部分 Linux 环境已经安装 g++，但不能假定所有发行版都自带。在 VSCode 的集成终端中先执行：

```
g++ -v
```

如果已经显示编译器版本信息，直接进入第 2 节；如果提示找不到命令，按自己的发行版选择一组安装命令。

Ubuntu / Debian：

```
sudo apt update
sudo apt install build-essential
```

Fedora：

```
sudo dnf install gcc-c++
```

等待安装完成，在 VSCode 终端中再次执行 `g++ -v`，确认可以显示 GCC 版本信息。

2. 在 VSCode 终端编译运行 Hello World

完成编译器安装后，需要完成本节验收。以下所有检查、编译和运行命令均在 **VSCode 的集成终端** 中执行。本环节手动编译运行，无需配置 `tasks.json`、`launch.json` 或安装 Code Runner。

1. 打开练习文件夹并检查编译器

在本机新建练习文件夹，例如 `cpp-hello`，通过 VSCode 的 **文件 → 打开文件夹（File → Open Folder）** 打开它，新建终端，执行：

```
g++ -v
```

确认终端显示编译器版本信息，没有“找不到命令”之类的错误。

2. 创建并保存 hello.cpp

在当前打开的文件夹中新建 `hello.cpp`，输入下面的代码并保存，也可以直接使用附带的 hello.cpp：

```
#include <iostream>

int main() {
    std::cout << "Hello World\n";
    return 0;
}
```

确保文件名是 `hello.cpp`，而不是 `hello.cpp.txt`，并且终端当前目录就是保存该文件的文件夹。如果文件刚修改过，先保存再编译。

3. 编译并运行

Windows（PowerShell）： 分别执行下面两行。第一行成功后再运行第二行。

```
g++ -std=c++17 hello.cpp -o hello.exe
.\hello.exe
```

macOS / Linux： 分别执行下面两行。第一行成功后再运行第二行。

```
g++ -std=c++17 hello.cpp -o hello
./hello
```

编译成功时通常没有输出；运行后应在同一个终端看到：

```
Hello World
```

如果报错找不到 `hello.cpp`，用 Windows 的 `Get-Location`、`dir` 或 macOS/Linux 的 `pwd`、`ls` 检查当前目录，切换到源码所在文件夹后重新编译。

4. 完成标准

☐ 能在 VSCode 的集成终端中执行 `g++ -v` 并看到编译器版本信息。

☐ 能在同一终端中用 `g++ -std=c++17` 成功编译 `hello.cpp`。

☐ 能运行生成的程序，并在终端看到 `Hello World`。